AirShield Corridor: Uso de Splines Cúbicos para Estimar PM2.5 en una Avenida Urbana

Juan Luis Ardila Velasco, Harry Esteban Sánchez Guevara, Pablo Enrique Quintero Callamand
Pontificia Universidad Javeriana

I. Resumen Ejecutivo

Hoy muchas ciudades reaccionan tarde a la contaminación. No necesariamente porque no tengan sensores, sino porque no tienen sensores suficientes para cubrir todos los puntos de una avenida. Entre un sensor y otro pueden existir zonas críticas que no se ven directamente, pero que sí afectan a las personas que caminan, viven o transitan por allí.

A partir de esta idea nace **AirShield Corridor**, un prototipo que usa análisis numérico para estimar la concentración de PM2.5 en lugares donde no hay sensores físicos. El sistema toma mediciones conocidas cada kilómetro y, mediante splines cúbicos, construye una curva que permite observar mejor cómo cambia la contaminación a lo largo de la avenida.

El proyecto fue implementado en Python usando Flask. La aplicación carga los datos desde un archivo `data.json`, genera una curva interpolada, muestra gráficas interactivas y calcula la exposición acumulada por sectores. También incluye una página de verificación donde se compara el método con un spline de Hermite, se estima incertidumbre mediante bootstrap y se evalúa el error con validación cruzada.

La propuesta de innovación de AirShield Corridor va más allá de mostrar una gráfica. La idea es que, en una versión futura, la ciudad pueda usar esta información para activar protocolos inteligentes como semáforos antismog, rutas limpias, alertas ciudadanas y prioridad para transporte público eléctrico en horas pico. En otras palabras, no se busca que la ciudad solo mida el problema, sino que pueda reaccionar ante él.

II. Introducción

La contaminación del aire es un problema cada vez más importante en las ciudades. Uno de los contaminantes más preocupantes es el PM2.5, que corresponde a partículas muy pequeñas suspendidas en el aire. De acuerdo con la Office of Environmental Health Hazard Assessment, el PM2.5 corresponde a partículas con diámetro de 2.5 micrómetros o menos, que pueden provenir de fuentes como vehículos, camiones, fábricas y procesos de combustión [2].

El problema es que medir la contaminación en toda una ciudad no es sencillo. Instalar sensores en cada punto sería costoso y difícil de mantener. Por eso, muchas veces las ciudades tienen sensores separados entre sí. Esto genera un vacío de información: se sabe qué ocurre exactamente donde está el sensor, pero no necesariamente qué pasa en los puntos intermedios.

En este proyecto se trabaja con una avenida que tiene mediciones de PM2.5 cada kilómetro. La pregunta principal es: ¿cómo se puede estimar la contaminación entre esos sensores? Para responderla se utiliza interpolación con splines cúbicos, una técnica de análisis numérico que permite construir una curva suave a partir de puntos conocidos.

La solución desarrollada se llama **AirShield Corridor**. Su objetivo es convertir mediciones aisladas en una visión más continua de la contaminación urbana. Así, el sistema puede ayudar a identificar zonas críticas que podrían pasar desapercibidas si solo se revisaran los sensores originales.

III. Descripción del Problema

El caso plantea una avenida con sensores ubicados cada kilómetro. Cada sensor mide la concentración de PM2.5 en ese punto específico. Sin embargo, entre dos sensores consecutivos también puede haber variaciones importantes de contaminación.

Los datos usados por el prototipo se encuentran en el archivo `data.json`. Estos datos representan la distancia desde el inicio de la avenida y la concentración de PM2.5 medida en cada punto.

Cuadro I
Datos originales de PM2.5

Distancia (km)	PM2.5 ($\mu g/m^3$)
0	45
1	52
2	68
3	85
4	92
5	88
6	79
7	63
8	48
9	42

A simple vista se observa que la contaminación aumenta hasta llegar al kilómetro 4, donde se registra el valor más alto, y luego empieza a disminuir. Sin embargo, los sensores solo muestran lo que ocurre en puntos exactos. Por eso, se necesita una forma de estimar qué pasa entre ellos.

IV. Idea General de la Solución

La solución consiste en tomar los datos de los sensores y construir una curva que represente el comportamiento aproximado de la contaminación a lo largo de toda la avenida.

En lugar de ver únicamente diez puntos separados, AirShield Corridor permite observar una trayectoria continua.

Esto hace posible estimar valores en puntos donde no hay sensores, por ejemplo, en el kilómetro 3.5, 4.2 o 6.7.

La idea principal puede resumirse así:

Mediciones de PM2.5 cada kilómetro
↓
Interpolación con splines cúbicos
↓
Curva continua de contaminación
↓
Identificación de sectores críticos
↓
Base para decisiones urbanas inteligentes

V. Propuesta de Mejora

AirShield Corridor es una idea que nace de una pregunta sencilla: ¿cómo puede una ciudad tomar decisiones inteligentes si no puede ver completamente lo que está ocurriendo con la contaminación entre un sensor y otro?

El prototipo actual puede resolver cierta parte importante de este problema: toma mediciones de PM2.5, construye una curva mediante splines cúbicos, muestra el comportamiento estimado de la contaminación y permite identificar sectores críticos. Sin embargo, la propuesta de mejora busca llevar esta base matemática un paso más allá.

La idea es que AirShield Corridor no sea únicamente una herramienta para visualizar datos, pues no generaría valor, sino una base para que la ciudad pueda reaccionar ante la contaminación. La idea es que cuando el sistema detecte una zona crítica, pueda activar un protocolo urbano inteligente compuesto por:

- **Semáforos antismog:** ajustar el flujo vehicular para reducir la entrada de carros hacia zonas con alta concentración de PM2.5.
- **Rutas limpias:** sugerir rutas alternativas que permitan evitar sectores con mayor exposición a contaminación.
- **Alertas ciudadanas:** informar a las personas cuando se encuentren cerca de una zona crítica.
- **Prioridad para transporte público eléctrico:** favorecer el paso de transporte público eléctrico en horas pico, reduciendo la presión sobre zonas contaminadas.

Esta propuesta se inspira en estrategias urbanas donde la movilidad se usa como herramienta para mejorar la forma en que se gestiona la ciudad. Por ejemplo, medidas como la congestión tarifada en la ciudad de Nueva York que muestra cómo las decisiones sobre tráfico pueden relacionarse con impactos en movilidad y calidad del aire [4].

En este sentido, AirShield Corridor busca demostrar que el futuro de una ciudad inteligente no depende únicamente de instalar más sensores, sino de tomar mejores decisiones con los datos que ya existen. El valor del proyecto está en convertir mediciones aisladas en una visión más completa del corredor urbano y, a partir de allí, abrir la posibilidad de activar respuestas inteligentes.

VI. Splines Cúbicos

VI-A. Fundamentos

La interpolación consiste en estimar valores desconocidos a partir de datos que sí se conocen. En este proyecto, los datos conocidos son las mediciones de PM2.5 tomadas por sensores ubicados cada kilómetro. El objetivo es estimar qué ocurre en los puntos intermedios, donde no hay sensores físicos.

Una forma sencilla de hacer esto sería unir los puntos con líneas rectas. Sin embargo, la contaminación no siempre cambia de manera lineal entre un punto y otro. Por eso, se eligieron los splines cúbicos, ya que permiten construir una curva suave que representa mejor la transición entre sensores.

Un spline cúbico funciona por tramos. Es decir, entre cada par de sensores consecutivos se construye un polinomio de grado tres. Luego, todos esos tramos se conectan de forma continua para formar una sola curva. Esto permite evitar cambios bruscos artificiales y obtener una representación más natural del comportamiento del PM2.5.

En el proyecto se utiliza la función `CubicSpline` de SciPy para construir esta interpolación. Esta función permite generar splines cúbicos a partir de datos discretos y evaluarlos en puntos intermedios [5].

VI-B. Ecuaciones

Si se tienen los puntos:

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n), \tag{1}$$

donde x_i representa la distancia en kilómetros y y_i representa la concentración de PM2.5, se busca construir una función $S(x)$ que pase por los puntos conocidos:

$$S(x_i) = y_i. \tag{2}$$

En un spline cúbico, cada tramo entre x_i y x_{i+1} se representa mediante un polinomio cúbico:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3. \tag{3}$$

Esto significa que no se usa una sola ecuación para toda la avenida, sino varias ecuaciones pequeñas, una por cada tramo entre sensores.

Para que la curva sea suave, los tramos deben cumplir tres condiciones principales:

- Pasar por los puntos originales.
- Unirse sin saltos entre un tramo y otro.
- Mantener continuidad en la pendiente y en la curvatura.

En el contexto de AirShield Corridor, estas condiciones permiten que la curva de PM2.5 represente un cambio gradual de la contaminación a lo largo de la avenida.

VII. Implementación de la solución

La solución consiste en una pagina web atractiva desarrollada en Python mediante el framework Flask, Los splines cubicos fueron implementados mediante la libreria SciPy [5].

Como ya se menciono, los datos se encuentran en formato Json, los cuales son leidos de la siguiente manera:

```
import os
from flask import Flask, render_template
import json
import numpy as np
from scipy.interpolate import CubicSpline,
CubicHermiteSpline

import plotly.graph_objs as go
import plotly.io as pio
import plotly.express as px
```

Lista de librerías y paquetes usados

```
x_coord = []
y_coord = []
```

Variables para almacenar los datos del Json

```
with open('data.json', 'r') as file:
    data = json.load(file)
```

Lectura del Json

```
puntos = data[0]["puntos"][0]
for x_json, y_json in puntos:
        x_coord.append(x_json)
        y_coord.append(y_json)
```

Guardar los datos en las variables

```
x = np.array(x_coord)
y = np.array(y_coord)
```

Convertir los datos en un arreglo de Numpy

```
sorted_indices = np.argsort(x)
x = x[sorted_indices]
y = y[sorted_indices]
```

Ordenar los datos como prerequisito para splines cubicos

VII-A. Pagina Principal

La pagina principal esta compuesta de un dashboard orientado al publico, donde pueden obtener información sobre los datos recopilados por los sensores.

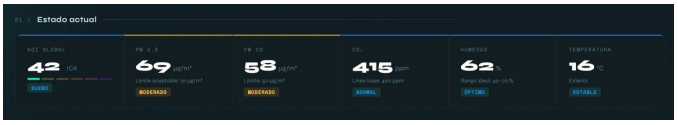

Figura 1. Header de la pagina principal con KPIs

La figura 1 es el header de la pagina principal, contiene los diferentes KPIs relevantes que una persona esperaría encontrar en una pagina de este estilo. El KPI *PM 2.5* representa el promedio de los datos ingresados. Los otros KPIs contienen información quemada.

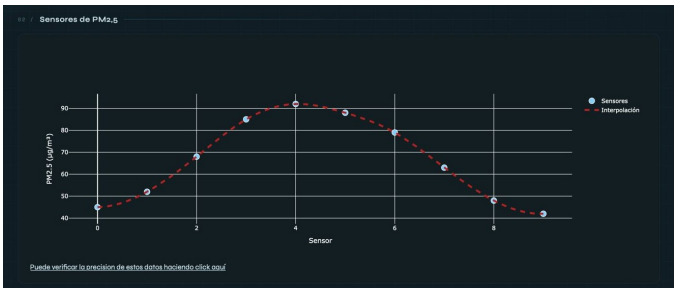

Figura 2. Datos de los sensores, la linea roja representa la interpolación por splines cúbicos. Note el enlace por el cual se accede a la pagina de verificación

La imagen anterior contiene la implementación principal de los splines cubicos para interpolar la información de los sensores, permitiendo obtener una estimación de la concentración de PM2.5 cada 0.1 km. Como ya se mencionó, esta implementación fue realizada mediante SciPy.

```
cs = CubicSpline(x,y)
```

Definir los splines cubicos con las variables

```
x_interp = np.arange(
np.min(x), np.max(x) + 0.1, 0.1)

y_interp = cs(x_interp)
```

Realizar la interpolación de los datos

```
spline_fig = go.Figure()

spline_fig.add_trace(go.Scatter(
    x=x, y=y,
    mode='markers',
    name='Sensores',
    marker=dict(
    color='LightSkyBlue', size=12,
    line=dict(
        color='white',
        width=1
        )
    )
)
)

spline_fig.add_trace(go.Scatter(
    x=x_interp, y=y_interp,
    mode='lines',
    name='Interpolacion',
    line=dict(
        color='firebrick',
        dash='dash',
        width=4
    )
))
```

Crear la visualización gráfica por medio de Plotly

```
        bar_fig.update_layout(
        plot_bgcolor="#121e23",
        paper_bgcolor="#121e23",
        font=dict(color="white")
    )
```

Diseño de la gráfica, todas las gráficas de la pagina siguen este estilo

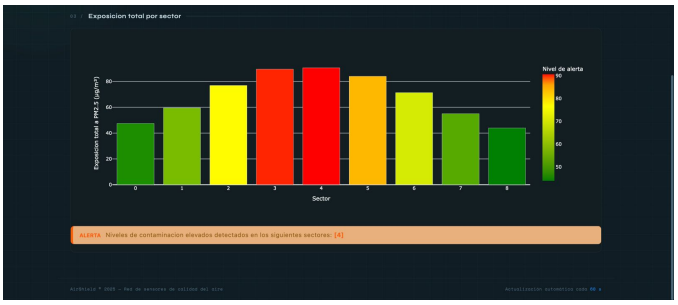

Figura 3. Concentración total de PM2.5 por sector

La siguiente gráfica representa la concentración de PM2.5 en cada *sector*, osea, cada tramo de 1Km entre sensores. Esto fue logrado mediante las integrales de cada traza entre sensores para identificar la concentración total (el área bajo la curva entre los puntos *A* y *A-1*). Se definió un gradiente de acuerdo con la concentración total, donde los valores por debajo de 70 son representados en verde, por debajo de 90 en naranja y superiores a 90 en rojo. Adicionalmente, se muestra una alerta que incluye cada tramo que esté por encima de 90.

```
danger = []
```

Variable para guardar los sectores que estén por encima de 90

```
F = cs.antiderivative()
integrals = np.diff(F(x))
```

Calcular las integrales de cada traza

```
print("integrales")
for i, integral in
enumerate(integrals):
    print(integral)
    if integral >= 90:
        danger.append(i)
```

Imprimir en consola las integrales y almacenarlas en el arreglo correspondiente en caso de que sea superior a 90

```
pmavg =
round(sum(integrals)/len(integrals))
```

Obtener el promedio de las integrales, este valor es que aparece en el header de la pagina

```
bar_segs=[0,1,2,3,4,5,6,7,8]

bar_fig = px.bar(
    x=bar_segs, y=integrals,
    labels={"x": "Sector",
            "y": "Exposicion total
            a PM2.5 (microg/m3)",
            "color": "Nivel de alerta"
            },
    color=integrals,
    color_continuous_scale=[
        (0.0, "green"),
        (0.7, "yellow"),
        (0.9, "orange"),
        (1.0, "red")
    ],
)
```

Crear la visualización gráfica junto con el gradiente

VII-B. Pagina de Verificación

Esta pagina contiene la verificación de los splines cúbicos mediante diferentes métodos, junto con visualizaciones similares a las de la pagina principal.

VII-B1. Splines Cúbicos de Hermite: La primera gráfica compara la interpolación por splines cubicos contra la interpolación por splines cubicos de hermite.

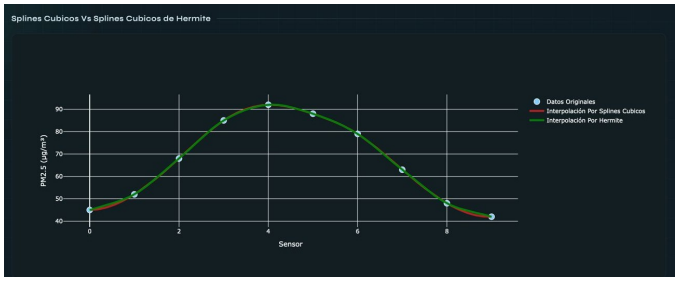

Figura 4. Comparación de splines cúbicos contra splines cúbicos de Hermite. La linea verde representa splines cúbicos mientras que la linea roja representa Hermite

Como se puede ver, son esencialmente idénticos, a excepción de los puntos frontera. Esto significa que las derivadas generadas por los splines cúbicos coinciden con los que la interpolación de Hermite espera, indicando que la información es consistente.

```
dydx = np.gradient(y, x)
hermite =
CubicHermiteSpline(x, y, dydx)
```

Obtener las derivadas de los puntos y definir la interpolación de hermite con estos

```
x_hermite = np.linspace(
np.min(x),
np.max(x),
50)
y_hermite = hermite(x_hermite)
```

Construir la interpolación

```
spline_fig = go.Figure()

spline_fig.add_trace(go.Scatter(
    x=x, y=y,
    mode='markers',
    name='Datos Originales',
    marker=dict(color='LightSkyBlue',
        size=12,
        line=dict(color='white',
        width=1))
))

spline_fig.add_trace(go.Scatter(
    x=x_interp, y=y_interp,
    mode='lines',
    name='Interpolacion Por
    Splines Cubicos',
    line=dict(color='firebrick',
    width=4)
))

spline_fig.add_trace(go.Scatter(
    x=x_hermite, y=y_hermite,
    mode='lines',
    name='Interpolacion Por
    Hermite',
    line=dict(color='green', width=4)
))
```

Crear la visualización gráfica

VII-B2. Intervalos de confianza mediante Bootstrapping: La siguiente gráfica establece los intervalos de confianza de 2.5 % y 95 % mediante la técnica de Bootstrapping

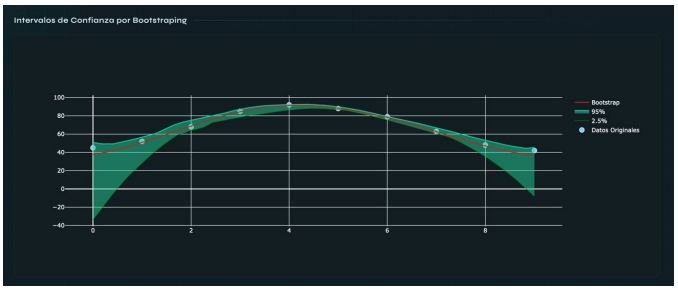

Figura 5. Intervalos de confianza de 2.5 % y 95 % de la interpolación

Como se puede ver, la incertidumbre del intervalo de 2.5 % aumenta drásticamente en los limites de la interpolación, este es un comportamiento esperado, dado que estos puntos contienen la menor información disponible para los splines, lo que se conoce como *efecto frontera, inestabilida de bordes o Fenómeno de Runge [1]*. Sin embargo, el intervalo de 95 % se mantiene muy cerca de la interpolación, incluso en los limites, indicando que la interpolación es bastante acertada.

```
n_boot = 1000
bootstrap_curves = []

for _ in range(n_boot):
```

Establecer 1000 muestras para boostrapping, junto con el arreglo para almacenar las curvas generadas

```
    idx = np.random.choice(len(x),
    len(x),
    replace=True)

    x_boot = x[idx]
    y_boot = y[idx]
```

Escoger un punto aleatorio dentro de la interpolación para realizar la verificación

```
    order = np.argsort(x_boot)
    x_boot = x_boot[order]
    y_boot = y_boot[order]

    x_unique, unique_idx =
    np.unique(x_boot, return_index=True)

    y_unique = y_boot[unique_idx]
```

Ordenar los resultados y eliminar cualquier duplicado

```
    if len(x_unique) < 4:
        continue

    try:
        spline =
        CubicSpline(x_unique, y_unique)

        y_interp = spline(x_interp)

        bootstrap_curves.append(y_interp)

    except:
        pass
```

Asegurar que tras la eliminación de duplicados aun hayan suficientes puntos para construir splines cúbicos

```
    bootstrap_curves =
    np.array(bootstrap_curves)

    mean_curve =
    np.mean(bootstrap_curves, axis=0)
```

Guardar los resultados en la variable correspondiente y encontrar el promedio

```
    lower =
    np.percentile(bootstrap_curves,
    2.5, axis=0)

    upper =
    np.percentile(bootstrap_curves,
    95.0, axis=0)
```

Definir los intervalos de confianza

```
boostrap_fig = go.Figure()

    boostrap_fig.add_trace(go.Scatter(
        x=x, y=y,
        mode='markers',
        name='Datos Originales',
        marker=dict(color='LightSkyBlue',
        size=10,
        line=dict(color='white',
        width=1))
    ))

    boostrap_fig.add_trace(go.Scatter(
        x=x_interp, y=lower,
        mode='lines',
        line=dict(color='green', width=1),
        name="2.5 %"
    ))

    boostrap_fig.add_trace(go.Scatter(
        x=x_interp, y=upper,
        mode='lines',
        fill='tonexty',
        name="95 %"
    ))

    boostrap_fig.add_trace(go.Scatter(
        x=x_interp, y=mean_curve,
        mode='lines',
        line=dict(color='firebrick',
        width=2),
        name="Bootstrap"
    ))
```

Crear la visualización gráfica

VII-B3. Verificación de error por RMSE y LOOCV: La gráfica final encuentra el error de la interpolación por medio de RMSE [3] y verificación cruzada LOOCV [6] para encontrar la distancia entre los datos y la interpolación.

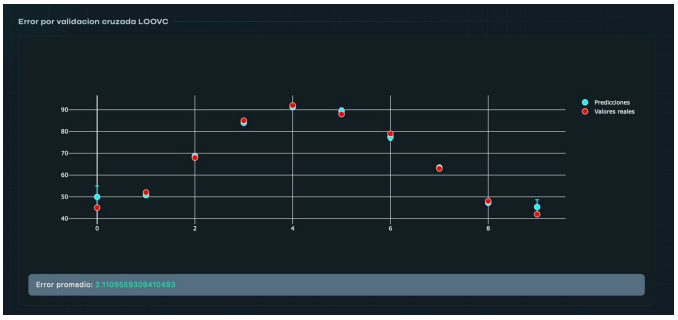

Figura 6. Interpolación junto con la validación cruzada LOOCV. Abajo de la gráfica se muestra el error promedio de la interpolación

Los resultados de la validación son bastantes similares a los obtenidos mediante Bootstrapping, se puede ver que la diferencia entre la interpolación y y la validación es bastante pequeña a excepción de los limites (por el ya mencionado Fenómeno de Runge). El error total para la interpolación es de aproximadamente 2.11, confirmando que la interpolación es bastante acertada al comportamiento esperado de los datos.

```
errors = []
predictions = []
```

Variables para almacenar los errores y las predicciones de LOOCV

```
for i in range(len(x)):
    x_train = np.delete(x, i)
    y_train = np.delete(y, i)

    spline = CubicSpline(x_train, y_train)

    y_pred = spline(x[i])

    predictions.append(y_pred)

    errors.append((y[i] - y_pred) ** 2)
```

Algoritmo LOOCV para splines cúbicos. Al final se guarda el error y la predicción

```
predictions = np.array(predictions)

rmse = np.sqrt(np.mean(errors))
abs_errors = np.abs(y - predictions)
```

Guardar las predicciones y encontrar el error promedio mediante RMSE

```
error_fig = go.Figure()

error_fig.add_trace(
    go.Scatter(
        x=x,
        y=predictions,
        mode="markers",
        name="Predicciones",
        marker=dict(color='cyan',
        size=12,
        line=dict(color='white',
        width=1)),
        error_y=dict(
            type="data",
            array=abs_errors,
            visible=True
        )
    )
)

error_fig.add_trace(
    go.Scatter(
        x=x,
        y=y,
        mode="markers",
```

```
            name="Valores reales",
            marker=dict(color='red',
            size=12,
            line=dict(color='white',
            width=1))
        )
    )
```

Crear la visualización gráfica

VII-C. Resultados

Los resultados de la verificación muestran que la interpolación mediante Splines cúbicos representa de forma correcta el comportamiento general de los datos. La comparación con el spline cúbico de Hermite muestra curvas muy similares, especialmente en la parte central del intervalo. Esto indica que la tendencia general de la información es consistente entre ambos métodos.

En el análisis mediante bootstrap se observa que la curva promedio mantiene una forma cercana a la interpolación original. La incertidumbre aumenta principalmente en los extremos del intervalo, lo cual es esperable porque en esas zonas hay menos información alrededor para construir la curva.

Por último, la validación cruzada permitió evaluar qué ocurre cuando se elimina un sensor y se intenta predecir su valor usando los sensores restantes. El error promedio obtenido mediante RMSE fue aproximadamente de 2.11, lo que indica que la interpolación tiene un comportamiento razonable frente a los datos del caso.

En conjunto, estas verificaciones muestran que el modelo no solo genera una curva visualmente atractiva, sino que también ofrece una estimación coherente del comportamiento de PM2.5 en una avenida.

VIII. Conclusiones

El proyecto AirShield Corridor permitió aplicar un método de análisis numérico a un problema urbano real de estimación de contaminación en zonas donde no hay sensores físicos. A partir de mediciones de PM2.5 tomadas cada kilómetro, se construyó una curva mediante splines cúbicos que permite observar mejor el comportamiento del contaminante a lo largo de la avenida.

El prototipo desarrollado en Python y Flask cumple con el objetivo principal de transformar datos aislados en una visualización continua. Además, el dashboard permite ver los sensores originales, la interpolación, la exposición acumulada por sectores y las zonas donde se detectan niveles elevados de PM2.5.

La integración del área bajo la curva por sectores aporta un elemento adicional importante, porque no solo muestra el valor puntual de contaminación, sino que permite analizar la exposición acumulada en cada tramo. Con esta información, el sistema identifica como crítico el sector entre los kilómetros 4 y 5, lo cual coincide con la zona donde se presenta el mayor valor de PM2.5 en los datos originales.

La página de verificación fortalece el proyecto, ya que compara el spline cúbico con Hermite, aplica bootstrap para observar la incertidumbre y utiliza validación cruzada Leave-One-Out para medir el error. El RMSE aproximado de 2.11 muestra que la interpolación se ajusta de forma razonable a los datos del caso.

Finalmente, AirShield Corridor demuestra que las matemáticas pueden convertirse en una herramienta para apoyar decisiones urbanas. Aunque el prototipo actual no controla semáforos ni envía alertas reales, sí construye la base matemática necesaria para una futura ciudad inteligente capaz de activar semáforos antismog, rutas limpias, alertas ciudadanas y prioridad para transporte público eléctrico. La idea central del proyecto es que una ciudad no necesita únicamente más sensores, sino mejores formas de usar los datos que ya tiene.

IX. Referencias

Referencias

[1] *Fenómeno de Runge*. 2023. URL: https://es.wikipedia.org/wiki/Fen%C3%B3meno_de_Runge (visitado 07-05-2026).

[2] Office of Environmental Health Hazard Assessment. *PM2.5*. CalEnviroScreen, State of California. Consultado en 2026. URL: https://oehha.ca.gov/calenviroscreen/indicator/pm25.

[3] *Raíz del error cuadrático medio*. 2026. URL: https://es.wikipedia.org/wiki/Ra%C3%ADz_del_error_cuadr%C3%A1tico_medio (visitado 07-05-2026).

[4] C. Tala Sánchez. *Movilidad urbana: impacto de la congestión tarifada en Nueva York*. El Ecosistema Startup. Consultado en 2026. 2025. URL: https://ecosistemastartup.com/movilidad-urbana-impacto-de-la-congestion-tarifada-en-nueva-york/.

[5] *SciPy*. 2026. URL: https://scipy.org/ (visitado 07-05-2026).

[6] Paco Sun. *Leave-One-Out Cross-Validation Explained*. 2025. URL: https://medium.com/@pacosun/one-out-all-in-leave-one-out-cross-validation-explained-409df5ff6385 (visitado 07-05-2026).